

House Price Classification using Random Forest on Kaggle Tabular Data

This project applies a Random Forest model to the Kaggle House Prices dataset by transforming continuous sale prices into categorical classes and performing a classification task on tabular housing data.

Overview

The task is based on the Kaggle House Prices dataset, where the goal is to predict housing prices using structured tabular features such as square footage, number of rooms, and property characteristics.

In this project, the regression problem was reformulated into a classification task by grouping house prices into three categories:

- Low (< \$150,000)
- Medium (\$150,000–\$300,000)
- High (> \$300,000)

The approach includes:

- Data cleaning and preprocessing (handling missing values, scaling, encoding)
- Feature engineering and transformation
- Training a Random Forest model
- Evaluating performance using RMSE and R^2

The model achieved reasonable predictive performance on the validation set, demonstrating that structured features can effectively capture housing price patterns.

Summary of Work Done

Data

- Type:
 - Input: CSV file with housing features (numerical + categorical)
 - Output: Categorical price class (0, 1, 2)
 - Dataset: Kaggle House Prices dataset
 - Size:
 - Approximately 1460 training samples
 - Approximately 80 features
 - Split:
 - 70% Training
 - 15% Validation
 - 15% Test
-

Preprocessing / Cleanup

- Converted SalePrice into 3 classes:

- 0 to Low
 - 1 to Medium
 - 2 to High
 - Handled missing values:
 - Numerical to median imputation
 - Categorical to the most frequent value
 - Feature scaling:
 - StandardScaler applied to numerical features
 - Encoding:
 - One-hot encoding for categorical variables
 - Removed unnecessary columns:
 - ID column dropped
-

Data Visualization

- Histogram of GrLivArea across price classes showed:
- Larger homes tend to fall into higher price classes
- Before/after scaling plots confirmed normalization worked

Key insight:

- The GrLivArea is strongly correlated with the price category
-

Problem Formulation

- Input: Housing features
- Output: Price class

Model Used

- Random Forest Regressor

Why Random Forest

- Handles tabular data well
- Works with nonlinear relationships
- Robust to overfitting compared to single trees

Metrics

- RMSE
 - R² Score
-

Training

- Library: scikit-learn
- Environment: Jupyter Notebook
- Training steps:
- Train/validation/test split
- Model fit on training data
- Evaluation on the validation set

- Stopping Criteria:
 - Default Random Forest parameters
-

Performance Evaluation

To evaluate the model's effectiveness, we used two primary metrics: RMSE and R^2 Score. These metrics provide insight into how well the model predicts the housing price categories

Conclusions

- Random Forest performed well on tabular housing data
 - Feature preprocessing was essential
 - Converting the regression to classification simplified the problem
-

Future Work

- Try true classification models
 - Hyperparameter tuning
 - Feature selection to reduce dimensionality
 - Use original regression instead of binning prices
 - Try neural networks on tabular data
-

How to Reproduce Results

1. Download the dataset from Kaggle (House Prices competition)
 2. Place train.csv and test.csv in the project directory
 3. Run notebook:
 - Data preprocessing
 - Model training
 - Prediction generation
 4. Output:
 - submission.csv for Kaggle
-

Repository Structure

- Kaggle Tabular Data.ipynb
 - Main notebook with full pipeline

Example structure:

- preprocessing
 - visualization
 - training
 - submission generation
-

Software Setup

Required Libraries

- pandas
- numpy
- matplotlib
- sklearn

Install with - pip install pandas numpy matplotlib sklearn

Data

- Source: Kaggle House Prices Competition
 - Files
 - train.csv
 - test.csv
-

Training

Run all notebook cells sequentially

- Data cleaning
 - Feature engineering
 - Model training
-

Performance Evaluation

- Evaluated using
 - RMSE
 - R^2 score
 - Validation set used for model assessment
-

Citations

- Kaggle House Prices Dataset
- scikit-learn documentation